

Physics-Informed Machine Learning Methodology and Applications in Quality Science & Engineering

Shancong (Sean) Mou

Department of Industrial and Systems Engineering
University of Minnesota, Twin Cities

Adapted from course material for **IE 8534: Physics-Informed Machine Learning — Methodology and Applications**, taught at the University of Minnesota.

Due to time constraints, this tutorial covers only a small portion of the related literature. All credit goes to the original authors of the cited papers; any errors or misinterpretations are my own.

Outline

1. **Introduction and overview**
 - ▶ Why PIML; forward vs. inverse problems
 - ▶ A brief introduction to PDEs
2. **Physics-informed neural network (PINN)**
 - ▶ physics-informed equation solver
3. **Operator learning**
 - ▶ Fourier Neural Operator (FNO); DeepONet
4. **PIML for inverse problems**
 - ▶ A PDE-constrained optimization viewpoint

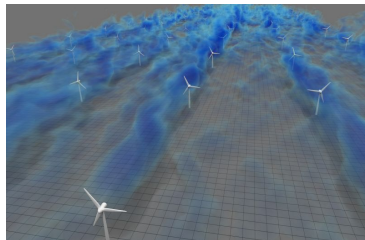
The world is governed by PDEs

PDEs govern physics at every scale:

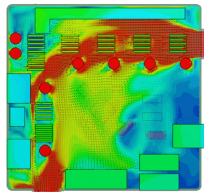
- ▶ **Large:** weather, wind farms
- ▶ **Medium:** gas turbines, 3D printing
- ▶ **Small:** chip cooling, blood flow

Simulate / design / control

⇒ *solve* or *invert* PDEs.



Wind farm (turbulence & bubbles, U. Twente)



Electronic-device thermal management (SimScale)

What is a PDE?

Solution $u(\vec{x})$, $\vec{x} = (x_1, \dots, x_n)$, parameterized by $\lambda \in \mathbb{R}^m$:

$$\underbrace{f(\vec{x}; u; u_{x_1}, \dots; u_{x_1 x_1}, \dots; \lambda) = 0}_{\text{governing equation, } \vec{x} \in \Omega}, \quad \underbrace{\mathcal{B}(u, \vec{x}) = 0}_{\text{boundary cond., } \vec{x} \in \partial\Omega}.$$

- ▶ $u : \mathbb{R}^n \mapsto \mathbb{R}$ — unknown function (n variables; scalar here).
- ▶ $u_{x_i} = \frac{\partial u}{\partial x_i}, \quad u_{x_i x_j} = \frac{\partial^2 u}{\partial x_i \partial x_j}, \dots$
- ▶ λ : coefficients, boundary/initial conditions.

Solve = find u satisfying f and $\mathcal{B}$.

Two problem types

Forward: fix $\lambda \rightarrow$ solve $u(\vec{x})$.

Traditionally FDM / FEM.

- + accurate, reliable
- expensive
- recompute per λ

Inverse: observe $u(\vec{x}) \rightarrow$ recover λ .

PDE-constrained optimization (adjoint).

- ▶ unknown coefficients / sources
- ▶ boundary / initial conditions
- ▶ spatio-temporal modeling

Learning helps: cheap *surrogate* $u = f_{\theta}(\vec{x}; \lambda)$ (or inverse $\lambda = f_{\theta'}(u)$); *physics* in the loss.

Example (forward): composite-material manufacturing

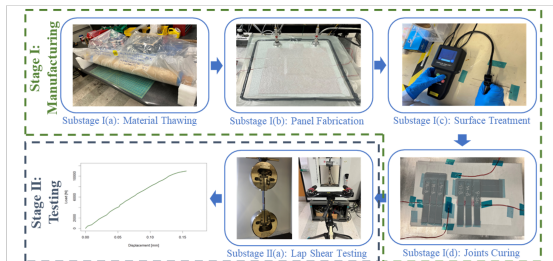


Fig. 1: Multi-stage manufacturing and testing processes of composite adhesive joints.

Task: predict bonding quality from process parameters.

Challenges:

- ▶ few samples (expensive)
- ▶ complex + heterogeneous data
- ▶ physics only *partially* known

Solution: physics-informed Neural ODE (Wang et al., 2024).

Linear vs. nonlinear

Linear: f linear in u and all derivatives.

Nonlinear — by where the nonlinearity sits:

- ▶ **Semilinear:** nonlinear in u ; linear in derivatives.
- ▶ **Quasilinear:** linear in the *highest-order* derivatives.
- ▶ **Fully nonlinear:** nonlinear in the highest-order derivatives.

Burgers' (viscous):

$$u_t + u u_x = \nu u_{xx} \quad \Rightarrow \quad \textit{quasilinear}.$$

nonlinear convection $u u_x$; linear in u_{xx} .

Order and classification

Order = highest derivative in f . Second-order workhorse:

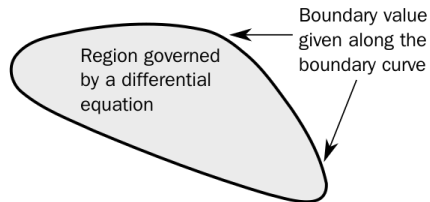
$$a u_{xx} + 2b u_{xy} + c u_{yy} + d u_x + e u_y + h u = g.$$

Discriminant $b^2 - ac$ classifies it:

- ▶ **Elliptic** (< 0): Laplace, $u_{xx} + u_{yy} = 0$
- ▶ **Parabolic** ($= 0$): diffusion, $u_t - D u_{xx} = 0$
- ▶ **Hyperbolic** (> 0): wave, $u_{tt} - c^2 u_{xx} = 0$

Burgers' $\rightarrow$ parabolic; Darcy $\rightarrow$ elliptic.

Boundary conditions



Boundary value problem (Wikipedia).

- ▶ **Dirichlet:** fix the value, $u|_{\partial\Omega} = g$.
- ▶ **Neumann:** fix the normal derivative, $\frac{\partial u}{\partial n}|_{\partial\Omega} = g$.
- ▶ **Robin:** $c_0 u|_{\partial\Omega} + c_1 \frac{\partial u}{\partial n}|_{\partial\Omega} = g$.
- ▶ **Cauchy:** Dirichlet *and* Neumann together.
- ▶ **Mixed:** different conditions on different parts of $\partial\Omega$.

How are PDEs solved?

- ▶ **Analytically:** closed-form — only simple PDEs / regular BCs.
- ▶ **Numerically:** FDM, FEM, spectral methods, ...

PIML question

Can a neural net represent u (or the solution *operator*)
— cheap, differentiable, data *and* physics?

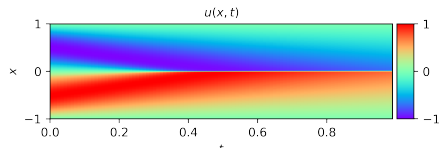
The rest of this tutorial: the recipes.

Two running PDE examples

1D viscous Burgers' equation

$$u_t + u u_x = \nu u_{xx}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$\text{IC: } u(x, 0) = -\sin(\pi x); \quad \text{BC: } u(\pm 1, t) = 0$$



Solution develops a sharp front (shock).

Steady-state 2D Darcy flow (heat transfer)

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in (0, 1)^2$$
$$u(x) = 0, \quad x \in \partial(0, 1)^2$$

- ▶ $a(x)$: heterogeneous permeability / conductivity
- ▶ $u(x)$: pressure / temperature field
- ▶ $f(x)$: source term

Linear, second-order *elliptic*.

Outline

Introduction and overview

A brief introduction to PDEs

Physics-informed neural network (PINN)

Operator learning

Fourier Neural Operator (FNO)

DeepONet

Physics-informed operator learning

Derivative-informed (Sobolev) training

PIML for inverse problems

Surrogate modeling: two routes

Goal: a cheap, differentiable surrogate for a PDE-governed system.

1. **PINN** — solve one PDE *instance*: $u = f_{\theta}(x, t)$ (this section)
2. **Operator learning** — learn a *family*, the map $a \mapsto u$ (FNO, DeepONet) (next section)

Single instance $\rightarrow$ learn the *operator* across instances.

Observation: the physics-informed loss (Raissi et al., 2019)

Write the PDE + BC/IC (parameterized by λ) as a single system:

$$(*): \quad u_t - \mathcal{L}[u; \lambda] = 0, \quad \mathcal{B}(u) = 0.$$

- ▶ **Solve** = find a solution that *satisfies* $(*)$.
- ▶ **Approximate** = find a solution that *minimizes the violation* of $(*)$.
- ▶ **Parameterize** the solution by a neural net $h(x, t; \theta) \Rightarrow$ a *training* problem: find weights θ that minimize the violation of $(*)$.

Solving a PDE $\longrightarrow$ training a neural network.

Two questions

We turned solving (*) into: *minimize the violation of (*) over the weights θ .*

1. How to **evaluate** the violation of (*)?
2. How to **minimize** it — i.e. how to **train**?

Q1 — how to evaluate the violation of (*)?

Strong form — residual *sampled* in the domain (interior + boundary):

$$R(x, t) = h_t - \mathcal{L}[h; \lambda] \text{ at sampled points} \quad \Rightarrow \quad \textbf{PINN}.$$

Weak form — residual *integrated* against test functions v :

$$\int R v \, dx \, dt = 0 \quad \Rightarrow \quad \text{Galerkin / Deep Ritz}.$$

In this tutorial, we focus on PINN.

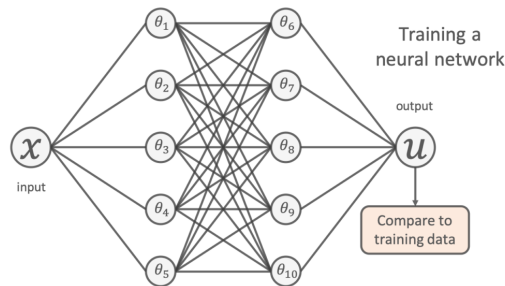
From residual to loss — network & objective

Network $h(x, t; \theta)$:

- ▶ input $(x, t) \in \mathbb{R}^2 \rightarrow$ output $u \in \mathbb{R}$
- ▶ derivatives h_t, h_x, h_{xx} from **automatic differentiation**

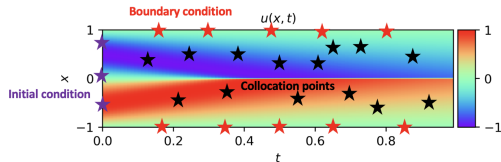
Objective — mean squared sampled residuals:

$$\min_{\theta} \frac{1}{N_f} \sum_{\text{interior}} R^2 + \frac{1}{N_b} \sum_{\text{bd./ic.}} R_{\partial}^2$$



Fully connected network h (benmoseley.blog).

Burgers' example — sampled points & loss terms



black interior, **red** boundary, **purple** initial points.

$$R = h_t + h h_x - \nu h_{xx}$$

$$\min_{\theta} \left[\frac{1}{N_f} \sum R^2 \right. \quad \text{interior} \\ \left. + \frac{1}{N_0} \sum (h(x, 0) + \sin \pi x)^2 \right. \quad \text{initial} \\ \left. + \frac{1}{N_b} \sum h(\pm 1, t)^2 \right] \quad \text{boundary}$$

colors match the sampled points.

Q2 — how to train? (it is hard)

Symptoms. stiff, slow, stalls; imbalanced loss terms; spectral bias; multi-scale / causal dynamics.

Several interpretations:

- ▶ gradient pathologies / NTK imbalance Wang et al., 2022
- ▶ **ill-conditioning** of the optimization De Ryck et al., 2024
- ▶ sharp, complex loss landscape Rathore et al., 2024

Tricks that help

1. **Respect causality** — train front-to-back in time. Wang et al., 2024
2. **Second-order optimizers** — L-BFGS fine-tune; Gauss–Newton / natural gradient. Rathore et al., 2024; Jnini et al., 2026
3. **Precondition** the operator when structure allows. De Ryck et al., 2024

Also: adaptive loss weighting, NTK-based balancing.

Take-home: physics as a loss term

Have data too? Just add a data term — same objective:

$$\hat{\theta} = \arg \min_{\theta} \underbrace{\frac{1}{N_d} \sum_{i=1}^{N_d} (u_{\text{obs},i} - h_i)^2}_{\text{data loss}} + \underbrace{\frac{1}{N_f} \sum_{i=1}^{N_f} R_i^2}_{\text{physics loss}}.$$

Pros

- ▶ physics in *one* extra loss term
- ▶ mesh-free; native to PyTorch / JAX

Cons

- ▶ trained per instance (fixed λ)

Fixed- λ limitation $\rightarrow$ learn the *operator*. Aside: PINN is a collocation / weighted-residual method, with the NN as basis.

Related variational solvers — Deep Galerkin, Deep Ritz — not covered here.

Outline

Introduction and overview

A brief introduction to PDEs

Physics-informed neural network (PINN)

Operator learning

Fourier Neural Operator (FNO)

DeepONet

Physics-informed operator learning

Derivative-informed (Sobolev) training

PIML for inverse problems

Operator learning: learn a family at once

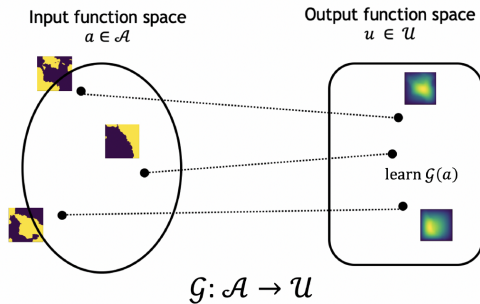
Each coefficient field a gives a solution u
— a map between *function spaces*

$$\mathcal{G}^* : \mathcal{A} \rightarrow \mathcal{U}, \quad a \mapsto u.$$

Goal. Learn $\mathcal{G}_\theta \approx \mathcal{G}^*$ over a distribution of inputs:

$$\min_{\theta} \mathbb{E}_{a \sim \mu} \|\mathcal{G}_\theta(a) - \mathcal{G}^*(a)\|.$$

Train once $\rightarrow$ evaluate on *any* new a .



Neural operator for Darcy flow (Li et al., 2021).

Two families: **Fourier Neural Operator (FNO)** and **DeepONet**.

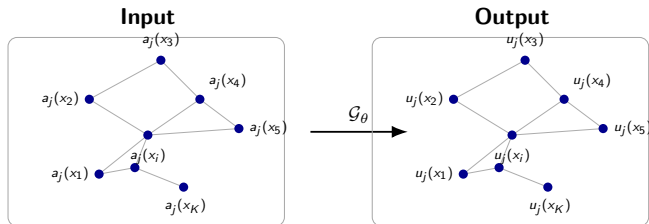
FNO: from functions to a graph

Darcy flow — given $a(x)$, solve $u(x)$; learn $u \approx \mathcal{G}_\theta(a)$:

$$-\nabla \cdot (\mathbf{a}(\mathbf{x}) \nabla \mathbf{u}(\mathbf{x})) = f(\mathbf{x}) \text{ on } (0, 1)^2, \quad u|_{\partial} = 0.$$

Functions are infinite-dimensional $\Rightarrow$ **discretize** on K points ($a_j, u_j \in \mathbb{R}^K$); learn a map between *graphs* $u_j \approx \mathcal{G}_\theta(a_j)$, trained on pairs $\{(a_j, u_j)\}_{j=1}^N$:

$$\min_{\theta} \frac{1}{N} \sum_{j=1}^N \|u_j - \mathcal{G}_\theta(a_j)\|_2^2.$$



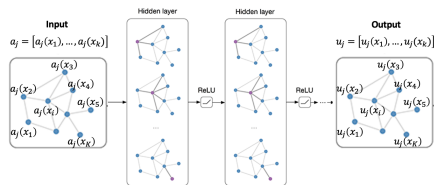
Graph Kernel Network

Lift $v_0(x) = a(x)$, apply a convolution-inspired layer (stack T), then **project** $v_T(x) \mapsto u(x)$:

$$v_{t+1}(x) = \sigma\left(Wv_t(x) + \int_D k_\phi(x, y, a(x), a(y)) v_t(y) dy\right).$$

Approximate the integral by **message passing** over neighbors $N(x)$:

$$\int_D k_\phi v_t dy \approx \frac{1}{|N(x)|} \sum_{y \in N(x)} k_\phi(e(x, y)) v_t(y), \quad e = (x, y, a(x), a(y)).$$



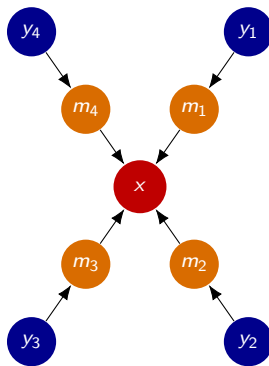
+ mesh- & geometry-independent
+ transfers across resolutions

Graph kernel network (Li et al., 2020).

GKN as message passing

One layer = gather messages from neighbors, then update node x :

$$v_{t+1}(x) = \sigma\left(Wv_t(x) + \frac{1}{|N(x)|} \sum_{y_i \in N(x)} m(y_i, x)\right), \quad m(y_i, x) = k_\phi(e(x, y_i)) v_t(y_i).$$



$m_i = m(y_i, x)$ — messages aggregated at x (Li et al., 2020).

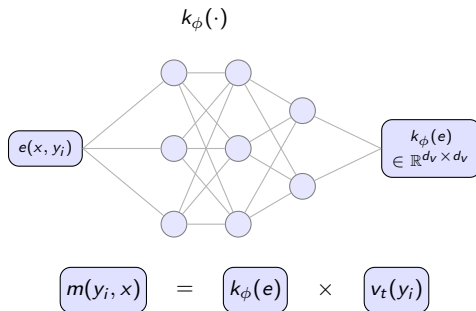
The kernel network k_ϕ

k_ϕ is a small MLP, *shared* across locations:

$$e(x, y_i) = (x, y_i, a(x), a(y_i)) \mapsto k_\phi(e) \in \mathbb{R}^{d_v \times d_v}.$$

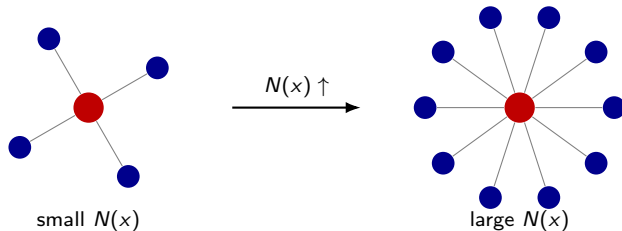
The message is a matrix–vector product:

$$m(y_i, x) = k_\phi(e(x, y_i)) v_t(y_i).$$



Graph Kernel Network: the bottleneck

Accuracy improves as the neighborhood $N(x)$ grows — it captures longer-range interactions:



But computation & storage scale with the *number of edges*, and accuracy is *sensitive* to $N(x)$ — large $N(x)$ gets expensive.

Idea: capture *global* interactions cheaply $\Rightarrow$ do the convolution in **Fourier** space (FNO).

Preliminary: Fourier transform & convolution theorem

Fourier transform — describe f by its frequencies:

$$\hat{f}(k) = (\mathcal{F}f)(k) = \int f(x) e^{-2\pi i x \cdot k} dx.$$

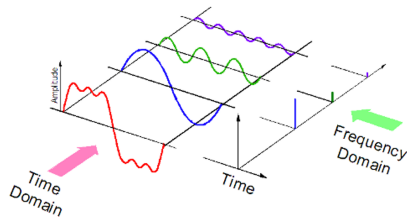
Convolution of g and h :

$$(g * h)(x) = \int g(\tau) h(x - \tau) d\tau.$$

Convolution theorem — convolution in space = multiplication in frequency:

$$(g * h)(x) = \mathcal{F}^{-1}\{(\mathcal{F}g) \cdot (\mathcal{F}h)\}.$$

Two tools: multiply in Fourier space, and keep only the low modes.



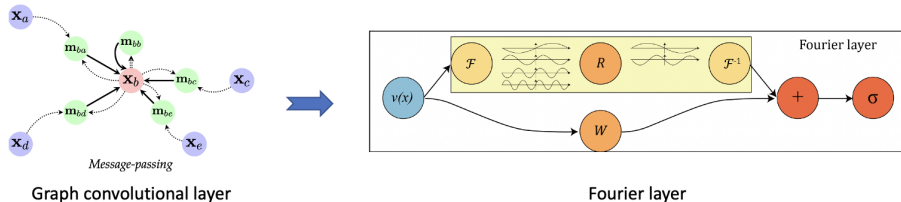
Time domain $\leftrightarrow$ frequency domain.

Fourier layer: convolution in Fourier space

Take a *translation-invariant* kernel $k_\phi(x-y) \Rightarrow$ the integral is a **convolution**. By the convolution theorem,

$$\int_D k_\phi(x-y) v_t(y) dy = \mathcal{F}^{-1}((\mathcal{F}k_\phi) \cdot (\mathcal{F}v_t)) = \mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}v_t).$$

Learn the kernel *directly* in the Fourier domain as R_ϕ .



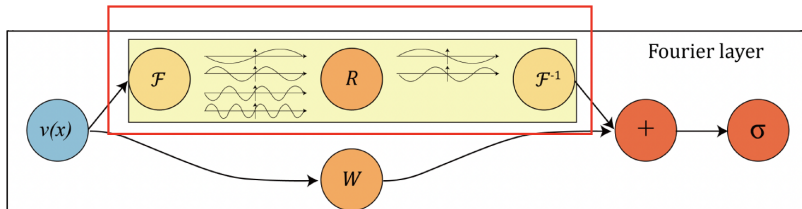
Fourier layer (Li et al., 2021).

Keep low modes $\rightarrow$ the FNO computational advantage

Dimensional reduction in the Fourier space: truncate to the $K_{\max}$ lowest Fourier modes ($K_{\max} \ll K$):

$$\mathcal{F}^{-1}(R_{\phi} \cdot (\mathcal{F}v_t)_{K_{\max}}).$$

FNO = lift $\rightarrow$ several Fourier layers (R_{ϕ} in spectrum + pointwise W) $\rightarrow$ project.
Cheap, and *resolution-free*.



The Fourier layer: algorithm

1. FFT each channel: $v_t \rightarrow \mathcal{F}v_t$.
2. Keep the lowest $K_{\max}$ modes; drop the rest.
3. Per mode k : linear mix $(R_\phi)^{(k)}(\mathcal{F}v_t)^{(k)}$, $(R_\phi)^{(k)} \in \mathbb{C}^{d_v \times d_v}$.
4. Inverse FFT to space; add pointwise Wv_t ; apply σ .

Darcy flow: results

Benchmarks vs. NN, RBM (POD),
FCN, PCANN, GNO, MGNO, LNO.

- ▶ FNO: lowest error
- ▶ *resolution-invariant* — train low,
test high

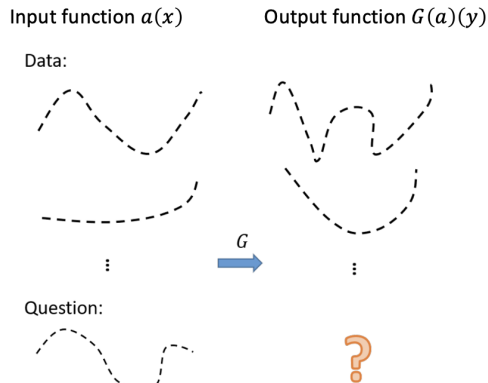
Network	$s=85$	$s=141$	$s=211$	$s=421$
NN	0.1716	0.1716	0.1716	0.1716
FCN	0.0253	0.0493	0.0727	0.1097
PCANN	0.0299	0.0298	0.0298	0.0299
RBM	0.0244	0.0251	0.0255	0.0259
GNO	0.0346	0.0332	0.0342	0.0369
LNO	0.0520	0.0461	0.0445	—
MGNO	0.0416	0.0428	0.0428	0.0420
FNO	0.0108	0.0109	0.0109	0.0098

Relative L_2 error at resolution s (Li et al., 2021).

Learn an operator from an input function $a(x)$ to an output function

$$u(y) \approx G(a)(y).$$

Same goal as FNO — a *different* architecture.



Input function $\rightarrow$ output function (Lu et al., 2021).

DeepONet: motivation problem

Darcy flow — given $a(x)$, solve $u(x)$:

$$-\nabla \cdot (\mathbf{a}(x) \nabla u(x)) = f(x) \quad \text{on } (0, 1)^2, \quad u|_{\partial} = 0.$$

Data — observation pairs $\{a^i, u^i\}$:

$$a^i = [a^i(x_1), \dots, a^i(x_m)] \in \mathbb{R}^m, \quad u^i = [u^i(y_1), \dots, u^i(y_K)] \in \mathbb{R}^K.$$

Aim: learn $u^i \approx G_{\theta}(a^i)$.

DeepONet: design

We learn an operator $G : a \mapsto G(a)(\cdot)$, where $G(a)$ is a *function*
 $G(a)(y) : y \in \mathbb{R}^d \mapsto \mathbb{R}$.

Given input a and *any* query point y ,
output $G(a)(y) \approx u(y)$.

- **Inputs:** a at sensors $\{x_1, \dots, x_m\}$, and the query y
- **Output:** $G(a)(y)$

Input: $\{a(x_1), a(x_2), \dots, a(x_m)\}$

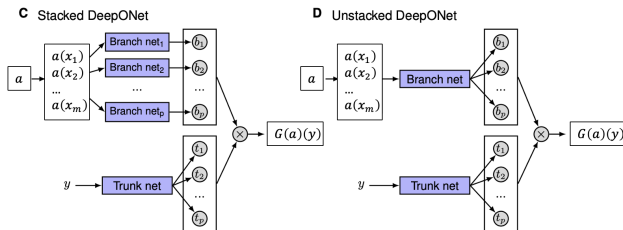


Idea: branch & trunk

Write $G(a)(y)$ as a function of y *conditioned* on a :

$$G(a)(y) = \sum_{k=1}^p \underbrace{b_k(a)}_{\text{branch}} \underbrace{t_k(y)}_{\text{trunk}}.$$

- ▶ $t_k(y)$: basis functions of the query y (**trunk net**)
- ▶ $b_k(a)$: coefficients from the input a (**branch net**)



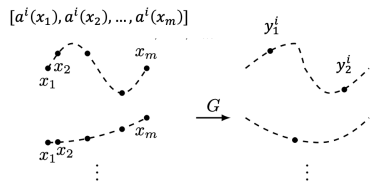
Branch–trunk architecture (Lu et al., 2021).

Loss & data structure

$$\mathcal{L}(\theta) = \sum_{i \in I} \sum_{j \in J_i} \|G_{\theta}(a^i)(y_j^i) - u^i(y_j^i)\|_2^2.$$

The i -th sample: input $(a^i, y_j^i) \rightarrow$ output $G_{\theta}(a^i)(y_j^i)$.

$$[\mathbf{a}, \mathbf{y}, \mathbf{G}(\mathbf{a})(\mathbf{y})] = \left[\begin{bmatrix} a^i(x_1), a^i(x_2), \dots, a^i(x_m), \\ \vdots \\ a^i(x_1), a^i(x_2), \dots, a^i(x_m), \\ \vdots \end{bmatrix}, \begin{bmatrix} y_1^i \\ \vdots \\ y_P^i \\ \vdots \end{bmatrix}, \begin{bmatrix} \mathbf{G}(\mathbf{a}^i)(y_1^i) \\ \vdots \\ \mathbf{G}(\mathbf{a})(y_P^i) \\ \vdots \end{bmatrix} \right]$$



Physics-informed operator learning

Idea. Make a neural operator $\mathcal{G}_\theta(a) \approx u$ physics-informed — add a PDE-residual loss on *designed* inputs a^{dsn} (no observations needed). For Darcy:

$$\mathcal{L} = \underbrace{\sum_n \|u_n - \mathcal{G}_\theta(a_n)\|_2^2}_{\text{data loss}} + \underbrace{\sum_j \lambda_1 \left\| -\nabla \cdot (\underbrace{a_j^{\text{dsn}}}_{\text{red}} \nabla \underbrace{\mathcal{G}_\theta(a_j^{\text{dsn}})}_{\text{blue}}) - f \right\|_2^2 + \lambda_2 \left\| \mathcal{G}_\theta(a_j^{\text{dsn}})|_{\partial} \right\|_2^2}_{\text{physics loss}}$$

- Physics loss is **cheap** — supervises any design pair $a_j^{\text{dsn}} \rightarrow \mathcal{G}_\theta(a_j^{\text{dsn}})$; can learn *even without* observation data.
- Difficult to balance physics loss with data loss.

Applies to both FNO and DeepONet.

Training neural operators in practice

- ▶ Neural operators are trained on **data** — paired $\{(a_j, u_j)\}$.
- ▶ **Pure physics-only training are challenging** for operators (unlike a PINN for one instance)
- ▶ As a **differentiable surrogate** (inverse problems, PDE-constrained optimization, design, control), the operator's **Jacobian** $\partial \mathcal{G}_\theta(a)/\partial a$ matters — not just the solution value.

$\Rightarrow$ *derivative-informed* (Sobolev-style) training.

What is Sobolev training?

Downstream tasks (PDE-constrained optimization, inverse problems) use the surrogate's *Jacobian*:

$$\min_f \Phi(f) = J(S(f), f), \quad \nabla_f \Phi = \underbrace{DS(f)^*}_{\text{Jacobian}} \nabla_u J + \nabla_f J.$$

A surrogate accurate in *value* but not in *Jacobian* gives wrong optimization directions.

Sobolev training — supervise value *and* Jacobian:

$$\mathcal{L}_{\text{Sob}} = \frac{1}{n} \sum_i \left[\|S_\theta(f_i) - u_i\|^2 + \mu \|DS_\theta(f_i) - DS(f_i)\|_F^2 \right].$$

Czarnecki et al., 2017.

Prior work: offline derivative-informed neural operators

DINO, DIFNO, SC-FNO:

1. Pre-generate Jacobian–vector products by solving the *forward sensitivity equation* r times per sample.
2. During training, match surrogate JVPs to the stored labels.

Smart bases cut the cost: KLE (inputs) + POD (outputs) $\Rightarrow$ only a few leading tangent directions per sample.

O'Leary-Roseberry et al. (2024); Yao et al. (2025); Behroozi et al. (2025).

Challenge: offline derivative-data generation is expensive

- ▶ Each sample: one forward solve + many linearized tangent solves (each $\sim$ a forward solve).
- ▶ Hours–days of compute; tens of GB stored per training set.
- ▶ Regenerate *from scratch* when data size, resolution, or sketch basis changes.

Can we make derivative-informed training on the fly?

No offline tangent solves, no stored labels, no architecture change.

Our idea: forward sensitivity equation as a physics loss

Differentiate $F(u, f) = 0$ at $u = S(f)$ with respect to f :

$$\underbrace{D_u F}_{\mathcal{A}} DS(f) + \underbrace{D_f F}_{\mathcal{B}} = 0,$$

with $\mathcal{A}, \mathcal{B}$ *known* tangent operators (from the PDE).

Replace DS by the surrogate $DS_\theta \Rightarrow$ residual

$$R_\theta(f) = \mathcal{A}(S_\theta(f), f) DS_\theta(f) + \mathcal{B}(S_\theta(f), f), \quad \mathcal{L}_{\text{raw}} = \mathbb{E}_f \|R_\theta(f)\|_F^2.$$

$S_\theta = S \Rightarrow R_\theta \equiv 0$: a *data-free* consistency condition for the Jacobian.

Making it online (a): random directional sketching

Problem. $DS_\theta(f) \in \mathbb{R}^{d_u \times d_f}$ needs $\mathcal{O}(d_f)$ passes per sample — not stochastic-trainable.

Sketch with random directions $v \sim \nu$ (one jvp call):

$$r_\theta(f, v) = R_\theta(f) v = \mathcal{A} DS_\theta(f)[v] + \mathcal{B} v, \quad \mathcal{L}_{\text{sTCL}} = \mathbb{E}_f \mathbb{E}_v \|r_\theta(f, v)\|^2.$$

Hutchinson: isotropic $v \Rightarrow \mathbb{E}_v \|R_\theta v\|^2 = \|R_\theta\|_F^2$ — enforces the sensitivity equation *in expectation*; full Jacobian never formed.

sTCL = the derivative-level analogue of PINN collocation.

Making it online (b): operator-aware conditioning

Raw residual $\|Aw + b\|^2$ has Hessian $H = 2A^\top A \Rightarrow \kappa(H) = \kappa(A)^2$: first-order optimizers *freeze* the important sensitivity modes.

Preconditioned loss $\ell_M = \langle Aw + b, M(Aw + b) \rangle$ with cheap M :

- ▶ **SPD elliptic** (nonlinear diffusion): DST/Fourier precondition., $\mathcal{O}(N \log N)$, no iteration.
- ▶ **Indefinite** (Helmholtz): $m=5$ MINRES target-matching, shifted-Laplacian.
- ▶ **Parabolic / advective** (Burgers): raw L^2 already adequate.

Results: accuracy vs. offline derivative-informed

3 PDEs $\times$ 3 data regimes; same FNO; $q=4$ random-Matérn probes both sides. (relative L_2 error,

	PDE	N	Pure FNO	sTCL (online)	Offline DI
mean $\pm$ std)	2D Helmholtz	128	16.11 ± 1.22	16.14 ± 2.00	16.51 ± 1.94
		512	12.31 ± 2.46	9.08 ± 0.60	7.91 ± 0.35
		1024	13.82 ± 3.56	8.06 ± 3.43	6.11 ± 0.46
	2D Nonlin. diffusion	128	10.67 ± 0.25	5.23 ± 0.05	4.04 ± 0.03
		512	5.39 ± 0.11	2.78 ± 0.10	2.48 ± 0.51
		1024	3.70 ± 0.14	2.05 ± 0.10	1.56 ± 0.16
	Burgers 1D	32	16.43 ± 1.77	10.63 ± 1.83	8.78 ± 0.44
		128	3.20 ± 0.29	2.27 ± 0.15	2.27 ± 0.18
		512	0.77 ± 0.11	0.65 ± 0.10	0.70 ± 0.08

sTCL is on the same accuracy scale as offline DI ($< 1.5\times$ every cell) — with *no* offline tangent-data pipeline.

Results: computing-time comparison

Per training job at $N=512$, 500 epochs, single GPU. *Offline-DI excludes the offline tangent-data generation stage.*

PDE	Pure FNO	sTCL (online)	Offline DI (training only)
2D Helmholtz	48 s	708 s (FMINRES-5)	509 s
2D Nonlin. diffusion	48 s	494 s (Fourier-precond.)	449 s
Burgers (1D)	102 s	536 s (raw L^2)	552 s

sTCL removes the offline precompute pipeline — no regeneration when data size, resolution, or sketch basis changes. (Per-step cost is comparable to offline-DI *training*.)

Derivative-informed training: takeaways

sTCL — a physics-informed way to add derivative information.

- ▶ Forward sensitivity equation enforced *in expectation* over sketched input directions (data-free).
- ▶ Operator-aware preconditioning makes first-order training effective.
- ▶ Comparable with offline derivative-informed accuracy — without the offline tangent-data pipeline.
- ▶ Plug-and-play for downstream PDE-constrained optimization / inverse problems.

Yang, Lu, Mou (2026).

Outline

Introduction and overview

A brief introduction to PDEs

Physics-informed neural network (PINN)

Operator learning

Fourier Neural Operator (FNO)

DeepONet

Physics-informed operator learning

Derivative-informed (Sobolev) training

PIML for inverse problems

Inverse problems: two cases

Given observations of the solution u , **recover unknowns** in the PDE.

- ▶ **Identify the PDE-governed system** — coefficients λ , sources, or the governing equation itself.
- ▶ **Identify unknown boundary / initial conditions.**

All as PDE-constrained optimization

Every such inverse problem has the same shape:

$$\min_{\lambda} J(u, \lambda) \quad \text{s.t.} \quad \underbrace{F(u, \lambda) = 0}_{\text{PDE} + \text{BC/IC}}.$$

- ▶ J : **cost** (objective) — e.g. data misfit $\frac{1}{2}\|u - u_{\text{obs}}\|^2$ (+ regularization)
- ▶ λ : coefficients / governing terms / BC-IC / controls
- ▶ constraint: the governing PDE must hold

The PDE is a **hard constraint**; u is determined by λ .

Classical: gradients via sensitivities

Eliminate $u = u(\lambda)$ and descend on $\Phi(\lambda) = J(u(\lambda), \lambda)$ — we need $\nabla_\lambda \Phi$.

- ▶ **Forward sensitivity:** differentiate the PDE $\Rightarrow$ one tangent solve *per parameter direction*. Costly when $\dim \lambda$ is large.
- ▶ **Adjoint:** one adjoint solve gives the *full* gradient — cost $\approx$ one extra PDE solve, independent of $\dim \lambda$.

The adjoint method

Lagrangian: $\mathcal{L} = J(u, \lambda) + \langle \psi, F(u, \lambda) \rangle$. Stationarity (KKT):

$$\underbrace{F(u, \lambda) = 0}_{\text{state}}, \quad \underbrace{(\partial_u F)^* \psi = -\partial_u J}_{\text{adjoint}}, \quad \nabla_\lambda \Phi = \partial_\lambda J + (\partial_\lambda F)^* \psi.$$

- ▶ accurate, rigorous — the standard for PDE-constrained optimization
- ▶ *intrusive*: needs a solver and hand-derived adjoint / derivatives

Where can PIML help?

PIML (1): relax to an unconstrained problem — PINN

Approximate u by a network $h(x, t; \theta)$ and make λ trainable; fold the PDE constraint into the loss (*soft*):

$$\min_{\theta, \lambda} \underbrace{\sum_i \|h(x_i, t_i; \theta) - u_{\text{obs}}\|^2}_{\text{data loss}} + \underbrace{\sum_i \|h_t - \mathcal{L}(h; \lambda)\|^2}_{\text{PDE loss}}.$$

- ▶ one joint optimization over (θ, λ) ; derivatives via autodiff
- ▶ no solver, no adjoint — but the PDE holds only *approximately*

Raissi, Perdikaris, Karniadakis (2019).

Example: recovering Navier–Stokes coefficients

Unknown $\lambda = (\lambda_1, \lambda_2)$:

$$\begin{aligned}u_t + \lambda_1(uu_x + vu_y) &= -p_x + \lambda_2(u_{xx} + u_{yy}), \\v_t + \lambda_1(uv_x + vv_y) &= -p_y + \lambda_2(v_{xx} + v_{yy}).\end{aligned}$$

Given scattered velocity data $\{(x_i, y_i, t_i, u_i, v_i)\}$, recover λ .

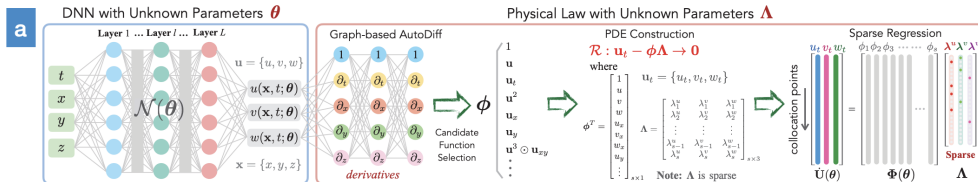
- ▶ net outputs stream function ψ and p ; $u = \psi_y$, $v = -\psi_x$ (incompressible)
- ▶ $\min_{\theta, \lambda} \sum_i (|u - u_i|^2 + |v - v_i|^2) + \sum_i (|f|^2 + |g|^2)$

Raissi, Perdikaris, Karniadakis (2019).

PINN for identifying governing equations

Express the unknown operator in a **library** $\phi = \{1, u, u^2, u_x, u_{xx}, \dots\}$ with *sparse* weights Λ , so $u_t + \Lambda^\top \phi = 0$:

$$\min_{\theta, \Lambda} \underbrace{\|h - u\|^2}_{\text{data}} + \alpha \underbrace{\|h_t - \Lambda^\top \phi\|^2}_{\text{physics}} + \beta \underbrace{\|\Lambda\|_0}_{\text{sparsity}}.$$



Chen, Liu, Sun (2021).

PIML (2): strictly enforce the PDE constraint

Soft penalties satisfy the PDE only approximately — enforce it.

(2.1) PINN-based hard constraints — hPINN. Turn the penalty into an **augmented Lagrangian**, so the PDE/BC are imposed as hard constraints (applied to inverse design):

$$\min_{\theta, \lambda} J + \langle \psi, r \rangle + \frac{\mu}{2} \|r\|^2, \quad r = \text{PDE residual},$$

with multipliers ψ and penalty μ updated as in the method of multipliers.

Lu, Pestourie, Yao, Wang, Verdugo, Johnson (2021).

Strict enforcement: bilevel & adjoint-mimicking (still PINN)

- ▶ **Bilevel PINN.** Inner level: solve the PDE with a PINN (constraint satisfied);
outer level: optimize λ via hypergradients (Broyden). Hao et al. (2023)
- ▶ **AONN.** Three networks — state, adjoint, control — in a direct–adjoint loop;
mimics the adjoint method, avoiding KKT penalty terms. Yin et al. (2024)

Both strictly enforce the PDE constraint — still PINN-based.

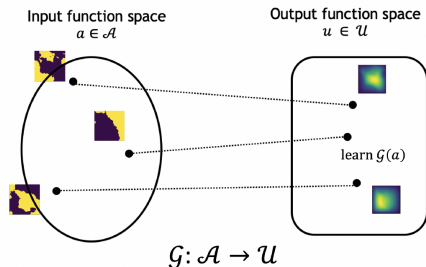
(2.2) Enforce the PDE via a neural operator

Use a pretrained operator $\mathcal{G}_\theta : a \mapsto u$ as a fast differentiable forward map, then **invert**:

$$\min_a \mathcal{L}_{\text{data}}(\mathcal{G}_\theta(a), u^*) + \mathcal{L}_{\text{pde}}(a, u^*) + R(a).$$

Or learn an inverse map $F : u \mapsto a$ and fine-tune on u^* .

Gradient flows through $\mathcal{G}_\theta \Rightarrow$ its **Jacobian** must be accurate (*Sobolev training*).



Inversion of neural operators (e.g. Wang et al., 2021).

Inverse problems: the landscape

Approach	PDE constraint	Notes
Classical adjoint	strict	accurate, intrusive (solver + adjoint)
PINN, relaxed	soft penalty	simple, joint (θ, λ) ; approximate
hPINN / bilevel / AONN	strict	PINN-based hard constraint
Neural-operator inversion	via surrogate	fast, amortized; needs accurate Jacobian

All are one PDE-constrained optimization — solved differently.

A few topics not covered

- ▶ **Spatio-temporal modeling of PDE-governed systems** — Neural ODE; learning continuous-time PDEs from sparse data.
- ▶ **Data + partial physics knowledge** — FEM-net.
- ▶ **Forward and inverse problems in a loop** — CNN + PDE in a loop; differentiable physics.

Thank you — Questions?



Shancong (Sean) Mou

Assistant Professor

Department of Industrial and Systems Engineering
University of Minnesota, Twin Cities

Email: mou00006@umn.edu

Phone: 404-200-5070

Web:

<https://sites.google.com/view/shancongmou>

Due to time constraints, this tutorial covers only a small portion of the related literature. All credit goes to the original authors of the cited papers; any errors or misinterpretations are my own.

Questions?

References I

-  M. Raissi, P. Perdikaris, G. E. Karniadakis. *Physics-informed neural networks*. J. Comput. Phys., 378:686–707, 2019.
-  S. Wang, Y. Teng, P. Perdikaris. *Understanding and mitigating gradient flow pathologies in PINNs*. SIAM J. Sci. Comput., 2021.
-  S. Wang, X. Yu, P. Perdikaris. *When and why PINNs fail to train: a neural tangent kernel perspective*. J. Comput. Phys., 449:110768, 2022.
-  S. Wang, S. Sankaran, P. Perdikaris. *Respecting causality for training physics-informed neural networks*. Comput. Methods Appl. Mech. Engrg., 421:116813, 2024.
-  T. De Ryck, F. Bonnet, S. Mishra, E. de Bézenac. *An operator preconditioning perspective on training in physics-informed machine learning*. ICLR, 2024.
-  P. Rathore, W. Lei, Z. Frangella, L. Lu, M. Udell. *Challenges in training PINNs: a loss landscape perspective*. ICML, 2024.

References II

-  A. Jnini, E. Kiyani, K. Shukla, J. F. Urban, N. Ahmadi Daryakenari, J. Müller, M. Zeinhofer, G. E. Karniadakis. *Curvature-aware optimization for high-accuracy physics-informed neural networks*. arXiv:2604.05230, 2026.
-  J. Sirignano, K. Spiliopoulos. *DGM: a deep learning algorithm for solving PDEs (Deep Galerkin)*. J. Comput. Phys., 375:1339–1364, 2018.
-  W. E, B. Yu. *The Deep Ritz method*. Commun. Math. Stat., 6:1–12, 2018.
-  Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar. *Neural operator: graph kernel network for partial differential equations*. arXiv:2003.03485, 2020.
-  Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar. *Fourier neural operator for parametric partial differential equations*. ICLR, 2021.
-  Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, A. Anandkumar. *Physics-informed neural operator for learning partial differential equations*. arXiv:2111.03794, 2021.

References III

-  L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis. *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*. Nature Machine Intelligence, 3:218–229, 2021.
-  S. Wang, H. Wang, P. Perdikaris. *Learning the solution operator of parametric partial differential equations with physics-informed DeepONets*. Science Advances, 7(40):eabi8605, 2021.
-  X. Yang, L. Lu, S. Mou. *Derivative-informed training of neural operators on-the-fly via sketched tangent consistency*. 2026.
-  W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, R. Pascanu. *Sobolev training for neural networks*. NeurIPS, 2017.
-  T. O’Leary-Roseberry, P. Chen, U. Villa, O. Ghattas. *Derivative-informed neural operator (DINO)*. J. Comput. Phys., 496:112555, 2024.
-  B. Yao, D. Luo, L. Cao, N. Kovachki, T. O’Leary-Roseberry, O. Ghattas. *Derivative-informed Fourier neural operator (DIFNO)*. arXiv:2512.14086, 2025.

References IV

-  A. Behroozi, C. Shen, D. Kifer. *Sensitivity-constrained Fourier neural operators (SC-FNO)*. ICLR, 2025.
-  Z. Chen, Y. Liu, H. Sun. *Physics-informed learning of governing equations from scarce data*. Nature Communications, 12:6136, 2021.
-  L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, S. G. Johnson. *Physics-informed neural networks with hard constraints for inverse design (hPINN)*. SIAM J. Sci. Comput., 43(6):B1105–B1132, 2021.
-  Z. Hao, C. Ying, H. Su, J. Zhu, J. Song, Z. Cheng. *Bi-level physics-informed neural networks for PDE-constrained optimization using Broyden's hypergradients*. ICLR, 2023.
-  P. Yin, G. Xiao, K. Tang, C. Yang. *AONN: an adjoint-oriented neural network method for all-at-once solutions of parametric optimal control problems*. SIAM J. Sci. Comput., 2024.